ABSTRACT

House price prediction project presents a machine learning-based web application for predicting house prices using **Linear Regression** and **Random Forest Regression** models. The system is designed to estimate property prices based on various features such as location, square footage, number of bedrooms, bathrooms, and other relevant parameters. The dataset is preprocessed to handle missing values, encode categorical variables, and normalize numerical features.

The dataset used for training and evaluation is sourced from publicly available real estate datasets (**Kaggle's House Price dataset**), and undergoes preprocessing steps such as handling missing values, one-hot encoding of categorical variables, and feature scaling. Feature selection techniques are applied to improve model performance and reduce overfitting.

Both **Linear Regression**, known for its simplicity and interpretability, and **Random Forest Regression**, known for its accuracy and ability to handle non-linear relationships, are trained and evaluated to compare performance. The trained models are serialized using **Joblib** and integrated into a user-friendly web interface built with **Flask**. Users can input property details through the web interface and receive instant price predictions.

The final application demonstrates a full machine learning pipeline—from data preprocessing and model training to deployment—highlighting the practical integration of data science in real-world decision-making systems, particularly in the real estate domain.

CONTENTS

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

1.1 Introduction

Data is at the heart of technical innovations, achieving any result is now possible using predictive models. Machine learning is extensively used in this approach. Machine learning means providing valid dataset and further on predictions are based on that, the machine itself learns how much importance a particular event may have on the entire system supported its pre-loaded data and accordingly predicts the result. Various modern applications of this technique include predicting stock prices, predicting the possibility of an earthquake, predicting company sales and the list has endless possibilities. Our aim is to predict a house price based on their needs and priorities. By analyzing previous market trends and price ranges, and also upcoming developments future prices will be predicted. The functioning involves a website which accepts customers specifications and then combines the application of neural network

1.2 Overview of Machine Learning

It is a subset of artificial intelligence (AI). It provides system the ability to automatically learn and improve by itself. It focuses on the development of computer programs that can access data learn by themselves. The process of learning begins with observations based on the examples that we provide. The aim is to make computers to learn by itself without the need of a human.

Machine learning can be classified into three types namely the supervised, unsupervised and reinforcement learning. Supervised machine learning algorithms can apply what has been learned in the past to new data predict future events. It analysis from a known training dataset, and produces a functions to predict outputs.

The system will provide outputs for inputs after training. The system will compare with the correct, intended output and find errors and modify it to make the model more practical and useful.

Semi-supervised machine learning algorithms is a combination of both supervised and unsupervised learning, In semi-supervised learning, an algorithm learns from a dataset that includes both labeled and unlabeled data, usually mostly unlabeled. Generally it is chosen when the sample data requires skilled resources in order to train from it. Otherwise, It doesn't require additional resources.

Reinforcement machine learning algorithms is a learning method that works based on feedback. Reinforcement learning differs from supervised learning in not needing labelled input/output pairs be presented. It is studied in various disciplines such as statistics, information theory etc.

1.3 Significance of Machine Learning

Machine learning has emerged as one of the most influential technologies in the modern data-driven world. It enables systems to automatically learn from historical data, identify patterns, and make intelligent decisions with minimal human intervention. Unlike traditional programming, where explicit instructions are required for each task, machine learning algorithms are designed to adapt and improve their performance over time as they are exposed to more data. This ability to learn and generalize makes machine learning highly effective in solving complex problems across various domains.

In particular, machine learning is revolutionizing industries such as healthcare, finance, e-commerce, and real estate by offering predictive insights and automating decision-making processes. For example, in the real estate industry, machine learning models can analyze vast amounts of historical property data to predict housing prices more accurately than traditional statistical methods. This not only saves time but also enhances the precision of price estimates, helping buyers, sellers, and real estate agents make more informed decisions.

Moreover, machine learning promotes innovation by enabling the development of intelligent applications such as recommendation systems, voice assistants, fraud detection systems, and autonomous vehicles.

1.4 Difficulties faced by previous models

While developing the house price prediction system using both Linear Regression and Random Forest Regression models, several challenges were encountered at different stages of the project. One of the primary difficulties was handling the quality and variability of the dataset. Real-world housing data often contains missing values, inconsistent formats, and categorical variables that require proper encoding. Ensuring clean, structured, and meaningful data was crucial, as both linear and ensemble models are sensitive to noise and irrelevant feature.

Another major challenge involved the limitations of the **Linear Regression** model. While it is computationally efficient and easy to interpret, it assumes a linear relationship between input features and the target variable. However, housing prices are often influenced by complex, non-linear factors such as location, neighborhood dynamics, and amenities, which Linear Regression fails to capture accurately. This led to underfitting and relatively poor prediction performance, especially in cases where property features did not have a strictly proportional impact on the price.

In contrast, the **Random Forest Regression** model offered better accuracy due to its ability to handle non-linear relationships and interactions among features. However, it introduced challenges of its own, particularly in terms of model complexity and size. Random Forest models tend to be large and difficult to interpret, which can make them harder to debug or optimize. Furthermore, saving and loading such models using **Joblib** sometimes resulted in increased memory usage and longer load times during deployment, especially on limited-resource environments.

During the **deployment phase using Flask**, integrating the machine learning pipeline with the web interface presented additional hurdles. Managing form inputs, ensuring correct data types, and validating user entries required careful backend logic. There were also challenges in maintaining consistency between the training feature set and the input features expected by the model at inference time. Finally, designing a responsive and user-friendly interface while handling errors gracefully was important to provide a smooth user experience.

CHAPTER 2

LITERATURE SURVEY

House price prediction has been a widely studied problem in the fields of data science, economics, and real estate analytics. Various statistical and machine learning models have been proposed and evaluated over the years to estimate property values based on a range of features such as location, size, age, and amenities.

[1] Sharma et al. (2024) introduced a machine learning framework for house price prediction using the **eXtreme Gradient Boosting (XGBoost)** algorithm, a powerful ensemble method known for its speed and high performance. The authors focused on leveraging the strengths of XGBoost—particularly its ability to **handle complex, non-linear relationships** and manage **missing or noisy data**, which are common in real estate datasets. XGBoost operates using gradient boosting on decision trees, where new models correct the errors of previous ones, leading to progressively better predictions.

In their study, Sharma et al. conducted **extensive hyperparameter tuning** to optimize the learning rate, maximum depth of trees, and the number of estimators. They employed techniques such as **grid search and cross-validation** to fine-tune these parameters, which significantly improved model generalization and reduced overfitting. Furthermore, they applied **structured feature engineering**, carefully selecting and transforming variables such as location, size, number of rooms, and property age into formats that XGBoost could effectively learn from.

The authors compared the performance of XGBoost with several baseline models, including Linear Regression, Decision Trees, and Random Forest. Their results showed that XGBoost achieved **the lowest root mean square error (RMSE)** and **highest R-squared values**, confirming its superior predictive capability. The model also produced **feature importance scores**, which helped identify the most influential variables affecting house prices—offering valuable insights to real estate professionals and policymakers.

[2] Nagaraja et al. (2011) conducted a foundational study on house price prediction using a time-series approach, emphasizing the use of autoregressive (AR) models to capture and analyze trends in the housing market. Their research demonstrated that house prices are not random but instead exhibit significant temporal dependencies—meaning that past prices have a direct

influence on future prices. By leveraging historical price data, the AR models could effectively learn and forecast these price patterns over time. The strength of their approach lay in its simplicity and interpretability, making it one of the earlier systematic efforts to incorporate time as a crucial factor in housing market analysis. This study played a significant role in shaping subsequent research, especially in motivating the development of more complex and accurate time-aware models, such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, in the deep learning era. By highlighting the importance of temporal patterns, Nagaraja et al.'s work laid the groundwork for integrating advanced sequence modeling techniques into real estate price forecasting.

[3] Das et al. (2020) introduced a novel, spatially-informed framework for house price prediction by leveraging **Geo-Spatial Network Embedding** techniques. Recognizing that geographical and topological factors play a critical role in determining property values, they modeled urban environments as **graph structures**, where each **node** corresponds to a property (or house) and **edges** represent spatial relationships such as proximity or neighborhood adjacency. This graph-based representation enabled the capture of complex spatial interdependencies that traditional tabular models often overlook.

By applying embedding techniques to this spatial graph, the model was able to learn **low-dimensional vector representations** of each property that encapsulated both local and global spatial context. These embeddings were then used as features in downstream predictive models, leading to a significant boost in accuracy, particularly in **high-density urban areas** where spatial heterogeneity is prominent. Their approach highlighted the potential of integrating **spatial graph learning** with real estate analytics, offering a more nuanced and context-aware method for predicting house prices compared to purely feature-based models. This work paved the way for subsequent research that combines **graph neural networks (GNNs)** with spatial data for enhanced performance in price estimation tasks.

[4] Chen et al. (2017) leveraged **Long Short-Term Memory (LSTM)** networks, a type of Recurrent Neural Network (RNN), for modeling the sequential nature of house price data over time. Recognizing that housing prices often exhibit long-range temporal dependencies—such as seasonal trends, economic cycles, or gradual urban development—the authors employed LSTM due to its capability to retain and utilize information from distant past time steps, addressing the vanishing gradient problem commonly faced by traditional RNNs.

Their methodology involved training LSTM models on **longitudinal datasets** containing historical price records, enabling the network to learn complex, nonlinear temporal patterns. Compared to conventional time-series approaches like Autoregressive Integrated Moving Average (ARIMA) or basic Autoregressive (AR) models, the LSTM-based model achieved **superior predictive accuracy**, particularly on datasets with extensive historical depth. This demonstrated the potential of deep learning models, not only in handling large-scale data but also in uncovering subtle and extended temporal dynamics that influence housing markets. The study by Chen et al. marked a significant shift towards more sophisticated temporal modeling techniques in real estate price forecasting, laying a foundation for future work incorporating both deep learning and hybrid models.

[5] Lu et al. (2017) introduced a **hybrid regression framework** designed to enhance house price prediction accuracy by combining the predictive power of multiple machine learning algorithms. Specifically, their approach involved **stacking** several base regressors, including **decision trees**—which are known for capturing nonlinear relationships and handling complex feature interactions—and **support vector regression (SVR)**, which excels at modeling data with high dimensionality and ensuring robust generalization through margin maximization.

By integrating these diverse models into a stacked ensemble, Lu et al. effectively harnessed the complementary strengths of each learner. The decision trees contributed flexibility and interpretability in capturing nonlinear patterns, while SVR provided strong generalization capabilities, especially in scenarios with limited or noisy data. Their experimental results demonstrated that such an ensemble approach could **reduce both bias and variance** errors more effectively than any individual model alone. This led to improved **generalization performance** on unseen data, underscoring the value of ensemble learning in mitigating overfitting and underfitting issues common in house price prediction tasks. The work of Lu et al. highlighted how combining multiple regressors in a principled manner can yield more reliable and accurate predictive models, influencing subsequent research on ensemble techniques in real estate analytics.

[6] Jain et al. (2020) performed a comprehensive experimental study comparing the performance of several widely-used machine learning algorithms for house price prediction, including **linear regression**, **decision trees**, and **random forests**. Using Python as the implementation platform, they rigorously evaluated these models across multiple datasets to assess their predictive

accuracy and robustness.

Their findings revealed that **random forests** consistently outperformed other methods, achieving the highest accuracy in predicting house prices. This superior performance was attributed to random forests' inherent ability to handle **high-dimensional feature spaces** and capture **complex nonlinear relationships** and interactions between variables without requiring extensive feature engineering. Unlike linear regression, which assumes a linear relationship between input features and the target variable, random forests use an ensemble of decision trees that aggregate diverse perspectives to reduce overfitting and improve generalization.

Jain et al.'s work provides valuable practical insights for researchers and practitioners entering the field of real estate price prediction, serving as a benchmark for selecting appropriate machine learning techniques based on empirical evidence. Their study emphasizes the importance of leveraging ensemble methods like random forests to effectively model the multifaceted and nonlinear nature of housing market data.

[7] Wang et al. (2018) proposed an innovative approach to house price prediction by leveraging **memristor-based artificial neural networks**, a cutting-edge technology inspired by the synaptic functions of the human brain. Memristors are neuromorphic devices capable of mimicking the plasticity and adaptive behavior of biological synapses, which enables them to perform computation and memory storage simultaneously at the hardware level. This represents a significant departure from traditional digital computing architectures and opens new possibilities for **energy-efficient**, high-speed machine learning applications.

In their work, Wang et al. developed a prototype system that utilized memristor arrays to implement neural network computations directly on hardware, bypassing the conventional bottlenecks associated with software-based models running on CPUs or GPUs. This hardware-level integration demonstrated not only the feasibility of real-time house price prediction but also a drastic reduction in power consumption, addressing key challenges in deploying AI models on edge devices or in resource-constrained environments.

The study marked a pioneering step toward **AI-on-chip** solutions tailored for smart real estate analytics, showcasing how neuromorphic computing can transform the scalability and sustainability of predictive modeling. By integrating memristor technology with neural network design, Wang et al.'s approach laid the groundwork for future research aimed at developing compact, efficient, and intelligent hardware systems capable of complex tasks.

[8] Wang et al. (2021) advanced the field of house price prediction by designing a **heterogeneous deep learning model** that effectively integrates multiple types of data—structured numerical features such as square footage, unstructured textual descriptions of properties, and visual information from images. Recognizing that real estate valuation relies on a diverse set of data modalities, their model sought to combine these complementary sources into a unified framework.

Central to their approach was the use of a **joint self-attention mechanism**, a powerful component originally popularized in natural language processing models. This mechanism enabled the model to dynamically assess and **assign different levels of importance to various input features** across the heterogeneous data types. By doing so, the network could focus more on the most relevant information—whether a critical phrase in the property description, a particular visual feature from images, or key numerical attributes—thereby enhancing predictive accuracy.

Beyond performance improvements, the self-attention framework also contributed to improved **interpretability**, allowing researchers and practitioners to better understand which features influenced the model's predictions and to what extent. This transparency is crucial in the real estate domain, where decision-makers value insights into the factors driving price estimations. Wang et al.'s work represents a significant step toward more holistic and explainable AI systems for housing market analysis, demonstrating the benefits of combining heterogeneous data and advanced attention-based modelling techniques.

[9] Liu and Liu (2019) introduced an enhanced approach to house price prediction by combining **Long Short-Term Memory (LSTM)** networks with a **Modified Genetic Algorithm (GA)** for automatic hyperparameter optimization. Traditional LSTM models often require extensive manual tuning of hyperparameters—such as learning rate, number of layers, and hidden units—to achieve optimal performance, a process that can be time-consuming and prone to suboptimal settings. To address this challenge, Liu and Liu integrated a modified GA to systematically explore the hyperparameter space and identify the best configurations.

This hybrid optimization strategy allowed their model to automatically adjust critical network parameters during training, leading to improved learning efficiency and predictive accuracy. When tested on extensive Chinese housing market datasets, the proposed method outperformed conventional LSTM models with manually selected hyperparameters, demonstrating superior

accuracy and robustness in forecasting house prices. The success of this approach underscored the potential of combining evolutionary algorithms with deep learning techniques, highlighting a promising direction for enhancing model performance through automated parameter tuning in complex real estate datasets.

[10] Banerjee and Dutta (2017) introduced a different perspective in housing market analysis by shifting from traditional regression-based price prediction to a **classification task**, focusing on predicting the **direction of house price movement**—whether prices would increase or decrease—rather than forecasting exact price values. This reframing is particularly relevant for stakeholders such as real estate investors and economists who prioritize understanding market trends and making strategic decisions based on price trajectories instead of precise numeric estimates.

To address this binary classification problem, the authors employed well-established machine learning algorithms, including **logistic regression** and **support vector machines (SVM)**. Their models were trained and tested on historical housing data, achieving **relatively high classification accuracy** in predicting the upward or downward movement of prices over a given time horizon. This approach demonstrated that effective trend prediction could be realized without the complexities of continuous value regression, offering a practical tool for market analysis and risk assessment.

Banerjee and Dutta's work is significant because it expands the scope of housing market analytics to include directional forecasting, enabling stakeholders to make more informed decisions about buying, selling, or holding properties based on anticipated market movements. Their study laid the groundwork for further research in classification-based market trend analysis, highlighting the value of classification models in real estate forecasting.

[11] Tan et al. (2017) proposed an innovative **Time-Aware Latent Hierarchical Model** to better capture the complex and evolving factors that influence house prices over time. Unlike traditional static models that treat housing market data as fixed snapshots, their approach models **latent variables**—hidden factors that affect prices but are not directly observed—and allows these variables to **evolve dynamically** as market conditions change.

The model leverages a **probabilistic graphical framework**, which explicitly integrates both **hierarchical geographic information** (such as neighborhoods, districts, and cities) and **temporal dependencies** across different time periods. This hierarchical structure reflects the

real-world nested nature of housing markets, where local and regional trends jointly impact price fluctuations. By incorporating temporal dynamics, the model adapts to shifts in market behavior, capturing trends such as gentrification, economic cycles, or policy changes over time.

Empirical results demonstrated that Tan et al.'s model significantly outperformed traditional static approaches, especially in **dynamic market conditions** where housing prices are influenced by continuously changing factors. This work underscored the importance of jointly modeling spatial hierarchy and temporal evolution to achieve more accurate and robust house price predictions, providing valuable insights for both researchers and practitioners dealing with volatile real estate markets.

[12] Madhuri et al. (2019) conducted a detailed comparative study of various regression techniques for house price prediction, focusing on **Linear Regression**, **Ridge Regression**, and **Lasso Regression**. Their analysis highlighted the distinct strengths and limitations of each method in the context of real estate data, which often involves numerous correlated features and potentially high-dimensional datasets.

They found that **Linear Regression** offers the advantages of simplicity, fast computation, and easy interpretability, making it a popular choice for baseline models. However, it suffers from sensitivity to **multicollinearity**, where strong correlations among predictor variables can inflate variance and degrade model stability. To address these challenges, Madhuri et al. evaluated **regularized regression techniques**—namely Ridge and Lasso regression—which incorporate penalty terms to constrain model complexity.

Their results demonstrated that both **Ridge Regression**, which applies an L2 penalty, and **Lasso Regression**, which uses an L1 penalty, outperform ordinary linear regression in scenarios with multicollinearity and high-dimensional feature spaces. Ridge regression effectively shrinks coefficients to reduce variance, while Lasso additionally performs feature selection by driving some coefficients to zero, resulting in sparser and more interpretable models. This study underscores the importance of regularization in improving prediction accuracy and model robustness in complex housing datasets.

[13] Madhuri et al. (2019) conducted an in-depth comparative study of several regression techniques applied to house price prediction, focusing on Linear Regression, Ridge Regression, and Lasso Regression. These methods are widely used in real estate analytics due to their interpretability and relative simplicity. The study emphasized the unique strengths and

limitations of each technique, particularly in the context of housing datasets that often contain numerous correlated features and high dimensionality.

Linear Regression, as the most basic approach, assumes a linear relationship between input features and house prices. It is popular because of its fast computation and ease of interpretation, making it a useful baseline model. However, Madhuri et al. noted that Linear Regression is sensitive to **multicollinearity**, where predictor variables are highly correlated. Multicollinearity inflates the variance of coefficient estimates, causing instability and reduced predictive reliability, which poses a significant challenge in complex housing datasets.

To address this, Madhuri et al. examined Ridge Regression, which introduces an L2 regularization penalty to the regression model. This penalty shrinks the size of coefficients but does not reduce any to zero, thus controlling model complexity and mitigating multicollinearity's negative effects. By balancing the fit to training data and penalizing overly large coefficients, Ridge Regression tends to improve generalization performance, especially when dealing with many correlated features.

In contrast, Lasso Regression applies an L1 penalty that encourages sparsity by driving some coefficients to zero, effectively performing feature selection. This property makes Lasso particularly useful when only a subset of variables are truly influential on house prices. Madhuri et al. found that Lasso not only helps in reducing model complexity but also enhances interpretability by identifying the most important features in the dataset.

Their results showed that both Ridge and Lasso outperformed standard Linear Regression in terms of accuracy and robustness, especially in high-dimensional or multicollinear scenarios. Ridge is preferable when many features contribute moderately, while Lasso excels in selecting a smaller set of impactful predictors. This study highlights the critical role of regularization techniques in developing stable, reliable, and interpretable models for housing price prediction.

CHAPTER 3

METHODOLOGY

3.1 Environment Setup

The environment setup marks the foundational stage of the house price prediction project, laying the groundwork for smooth development and deployment. This phase involves preparing the system with all the essential tools, libraries, and configurations needed to build a robust machine learning application. A properly configured environment helps ensure reproducibility, maintainability, and consistency across different systems and stages of the project.

Python was chosen as the programming language due to its simplicity, readability, and extensive ecosystem that supports data science, machine learning, and web development. Its widespread use in the industry and academia, along with a rich collection of libraries, makes it an ideal choice for developing end-to-end data-driven applications. To maintain clean and conflict-free project dependencies, a virtual environment was recommended. Using tools like venv or virtualenv, the workspace was isolated from system-wide packages, which helped prevent version clashes and allowed for more manageable dependency control.

A number of Python libraries were installed using the pip package manager to support various stages of the project. For data manipulation and numerical operations, Pandas and NumPy were used extensively, offering efficient data structures and functions for handling large datasets. To facilitate visual insights during the Exploratory Data Analysis (EDA) phase, Matplotlib and Seaborn were included. These visualization libraries enabled the creation of insightful charts and graphs that helped in understanding patterns, trends, and relationships within the data.

Scikit-learn, a powerful and comprehensive machine learning library, formed the core of the model development pipeline. It provided a wide range of tools for data preprocessing, feature selection, model training, and evaluation. In this project, Scikit-learn was used to implement algorithms such as Linear Regression and Random Forest, allowing for effective comparison and optimization. Additionally, Joblib was employed for efficiently saving and loading trained models and preprocessing pipelines, which streamlined the process of model reuse and deployment.

For deploying the trained machine learning model as a web application, the Flask web

framework was utilized. Flask is a lightweight and flexible micro-framework that allows rapid development of APIs and web interfaces. Its simplicity and modular nature made it particularly well-suited for integrating the machine learning model with a user-friendly web front end, enabling users to input features and receive real-time predictions.

All necessary libraries and tools were either installed manually via pip install commands (e.g., pip install flask pandas numpy scikit-learn seaborn matplotlib joblib) or managed through integrated development environments like Jupyter Notebook, VS Code, or PyCharm. Once the environment was fully configured and tested, the system was ready to move on to the next stages of the project, including data loading, analysis, model building, and deployment. This meticulous setup ensured that the development process would be smooth, scalable, and aligned with best practices in machine learning application development.

3.2 Importing Libraries

Importing the required libraries is the first coding step in any machine learning project, as it provides the necessary tools to perform data analysis, build models, visualize results, and deploy applications. In this house price prediction project, several Python libraries were imported to handle different tasks efficiently.

The **pandas** and **numpy** libraries were imported to support data manipulation and numerical operations. pandas makes it easy to load, inspect, and clean the dataset, while numpy is used for mathematical computations, especially when dealing with arrays and numerical transformations.

For visualization and exploratory data analysis, **matplotlib.pyplot** and **seaborn** were used. These libraries help generate plots such as histograms, boxplots, bar charts, and scatter plots, allowing a better understanding of data distribution and relationships between variables.

The **scikit-learn (sklearn)** library was a critical component of the project. It was used for splitting the data into training and testing sets (train_test_split), preprocessing categorical data using encoders like LabelEncoder and OneHotEncoder, and building predictive models with LinearRegression and RandomForestRegressor. Additionally, it was used for evaluating model performance using metrics like **MAE (Mean Absolute Error)**, **MSE (Mean Squared Error)**, **RMSE (Root Mean Squared Error)**, and **R^2 Score**.

The **ColumnTransformer** module from sklearn was particularly important for preprocessing,

as it allowed combining both numerical and categorical data transformations in a streamlined way. The **joblib** library was also imported to save and load machine learning models and preprocessing pipelines efficiently, making it suitable for deployment purposes.

Overall, importing the right set of libraries at the beginning ensured that all stages of the machine learning workflow—data loading, preprocessing, modeling, evaluation, and deployment—could be executed seamlessly. This modular approach to library usage also made the code more organized and scalable.

3.3 Data Loading and Inspection

After setting up the environment, the next essential step in the project was loading and inspecting the dataset. The dataset used in this project comprises house listings from Hyderabad, containing various features such as location, property type, area, building status, rate per square foot, and price in lakhs. To begin, the dataset was loaded using the pandas library, a robust tool for handling structured data in Python. The CSV file was read using the pd.read_csv() function, converting the raw data into a DataFrame—a table-like structure that makes it easy to manipulate and analyze.

```
Dataset shape: (3660, 7)

First 5 rows:
```

	Unnamed: 0	title	location	price(L)	rate_persqft	area_insqft	building_status
0	0	3 BHK Apartment	Nizampet	108.00	6000	1805	Under Construction
1	1	3 BHK Apartment	Bachupally	85.80	5500	1560	Under Construction
2	2	2 BHK Apartment	Dundigal	55.64	5200	1070	Under Construction
3	3	2 BHK Apartment	Pocharam	60.48	4999	1210	Under Construction
4	4	3 BHK Apartment	Kollur	113.00	5999	1900	Under Construction

```
Data types and missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3660 entries, 0 to 3659
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Unnamed: 0      3660 non-null   int64
 1   title           3660 non-null   object
 2   location        3660 non-null   object
 3   price(L)        3660 non-null   float64
 4   rate_persqft    3660 non-null   int64
 5   area_insqft     3660 non-null   int64
 6   building_status 3660 non-null   object
dtypes: float64(1), int64(3), object(3)
memory usage: 200.3+ KB
None
```

FIG 3.1 : Feature description of Actual dataset

Once the data was loaded, an initial inspection was performed to understand its shape and content. The .shape attribute was used to check the number of rows and columns, helping assess the dataset's size. The .head() method was applied to view the first few entries, giving a snapshot of the kind of data available. Next, the .info() function provided valuable details about each column, including data types and the presence of null or missing values. Identifying missing or incorrectly formatted values at this stage is important, as they can affect the model's performance if not handled properly.

Descriptive statistics were generated using the .describe() method, which summarizes numerical features like area_insqft, rate_persqft, and price(L) by providing their mean, standard deviation, minimum, and maximum values. This analysis helped detect potential outliers, skewed distributions, and inconsistencies in scale that would later influence feature engineering and model training.

The inspection also included observing categorical features such as title, location, and building_status to understand the diversity and distribution of property types, geographic spread, and construction statuses. This helped prepare for later steps involving encoding categorical variables into a machine-readable format. Overall, this phase ensured a comprehensive understanding of the dataset, laying a strong foundation for cleaning, exploration, and modeling.

3.4 Data Cleaning

Data cleaning is one of the most important steps in any machine learning project, as the quality of the data directly affects the performance of the model. In this house price prediction project, after the dataset was loaded and inspected, the cleaning phase was carried out to ensure that the data was accurate, consistent, and ready for analysis.

The first task was handling missing values. Rows containing null values were removed using the dropna() function, ensuring that the dataset used for training was complete and free from any undefined entries that could disrupt model learning. This step helped maintain the integrity of the input data and ensured that models wouldn't be trained on incomplete information.

Next, data transformation was performed to ensure uniformity in units. The price column in the dataset was originally represented in lakhs (price(L)), which was not suitable for fine-grained

calculations. To make the price more interpretable and accurate for predictions, it was converted into rupees by multiplying the value by 100,000. Furthermore, a new column called price_per_sqft was derived by dividing the total price by the area (area_insqft) to normalize price based on property size, which provided a more meaningful feature for the prediction model.

Categorical columns such as title, location, and building_status often had leading or trailing whitespace, which can lead to duplication of categories during encoding. These were cleaned using the .str.strip() function to standardize the values and avoid misrepresentation during preprocessing.

Through these steps, inconsistencies, missing data, and unit mismatches were eliminated, leading to a clean and structured dataset. This cleaning process ensured that the data fed into the machine learning models was of high quality, reducing the risk of errors and improving model reliability and accuracy.

	Unnamed: 0	title	location	price(L)	rate_persqft	\
0	0	3 BHK Apartment	Nizampet	108.00	6000	
1	1	3 BHK Apartment	Bachupally	85.80	5500	
2	2	2 BHK Apartment	Dundigal	55.64	5200	
3	3	2 BHK Apartment	Pocharam	60.48	4999	
4	4	3 BHK Apartment	Kollur	113.00	5999	

	area_insqft	building_status	price	price_per_sqft
0	1805	Under Construction	10800000.0	5983.379501
1	1560	Under Construction	8580000.0	5500.000000
2	1070	Under Construction	5564000.0	5200.000000
3	1210	Under Construction	6048000.0	4998.347107
4	1900	Under Construction	11300000.0	5947.368421

FIG 3.2 : Feature description of dataset after data preprocessing

3.5 Exploratory Data Analysis

3.5.1 Univariate Analysis

Univariate analysis is a fundamental part of exploratory data analysis (EDA) that focuses on examining the distribution and behaviour of individual variables one at a time. In the house price

prediction project, univariate analysis was conducted to understand the characteristics of key features in the dataset before building predictive models. This step helped identify patterns, outliers, and skewed distributions that could influence the model's accuracy.

One of the initial variables explored during the analysis phase was the property type, referred to as the **title** in the dataset. To gain insights into the distribution of different property types, a count plot was created. This visualization illustrated how often each category—such as 2 BHK, 3 BHK Villa, Residential Plot, and others—occurred within the dataset.

By analyzing the frequency of each property type, the plot provided a clear picture of the most commonly listed real estate options. This helped in identifying prevailing trends in housing availability, as well as gauging customer preferences based on supply. For instance, a significantly higher number of 2 BHK properties might suggest strong demand in that segment, or simply reflect a larger supply of such homes in the market. These insights not only supported better understanding of the dataset but also informed modeling decisions by highlighting dominant categories that could influence price prediction.

Next, the **price distribution** was examined using a histogram. Since housing prices often vary widely and contain outliers, a histogram helped visualize the central tendency and spread of property prices. To avoid skewing the visual results due to extreme values, the upper limit was capped at the 95th percentile. This provided a clearer view of the bulk of the data and helped in making decisions such as normalization or log transformation if required.

Similarly, the **area of the properties** (area_insqft) was explored. Another histogram was used to study the distribution of property sizes, which helped identify whether the dataset was dominated by small apartments, large houses, or plots. Like the price analysis, the extreme values were excluded to better observe the overall pattern.

The univariate analysis also included an investigation of the **price per square foot**, a derived metric that indicates how much each unit area costs. This feature is especially useful for comparing the cost-efficiency of properties of different sizes and types. A histogram of this feature helped assess variability and detect unusually high or low values.

Overall, univariate analysis provided a clear understanding of individual features in the dataset. It laid the foundation for more complex multivariate analysis by helping to identify which

features had meaningful variation, which were heavily skewed, and which might need to be transformed or scaled. It also helped detect any anomalies that might distort model training if not addressed early.

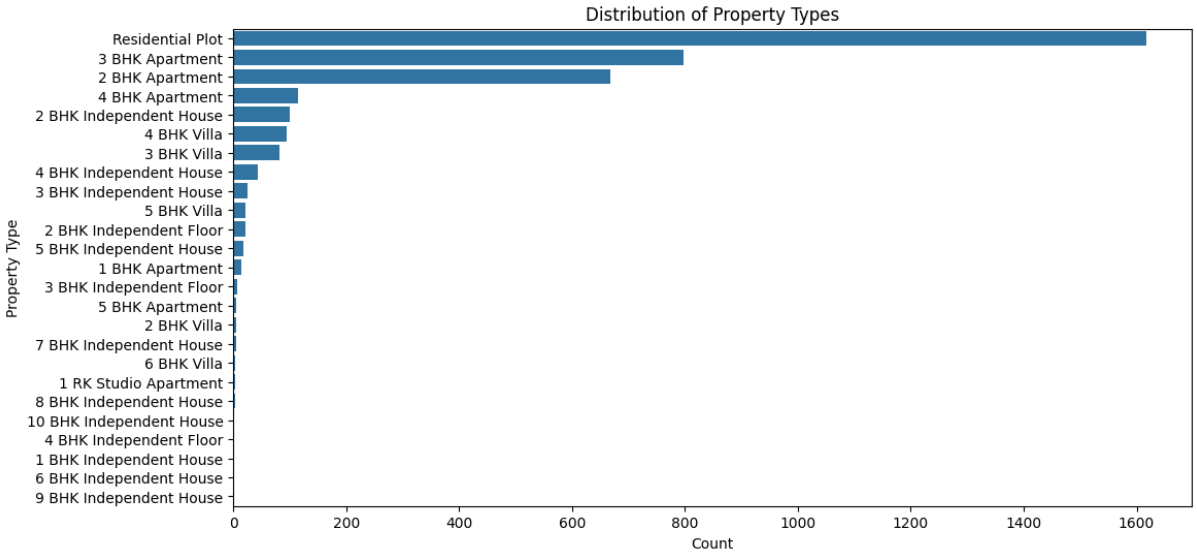

FIG 3.3 : Distribution of Property Types

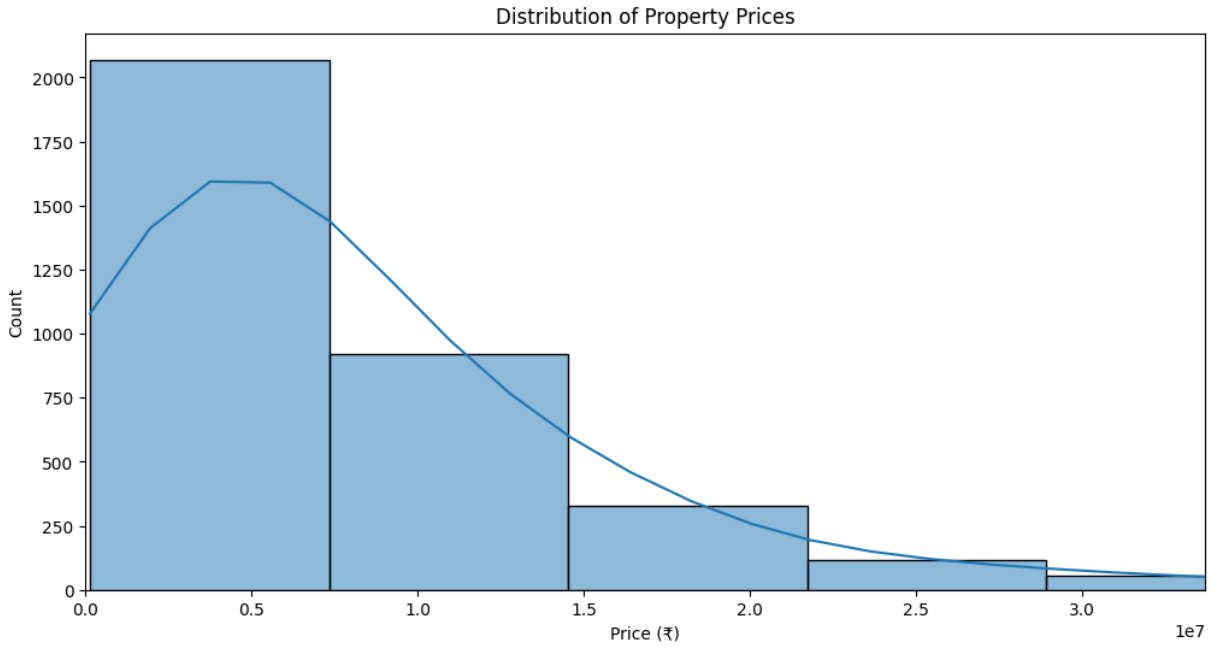

FIG 3.4 : Distribution of Property Prices

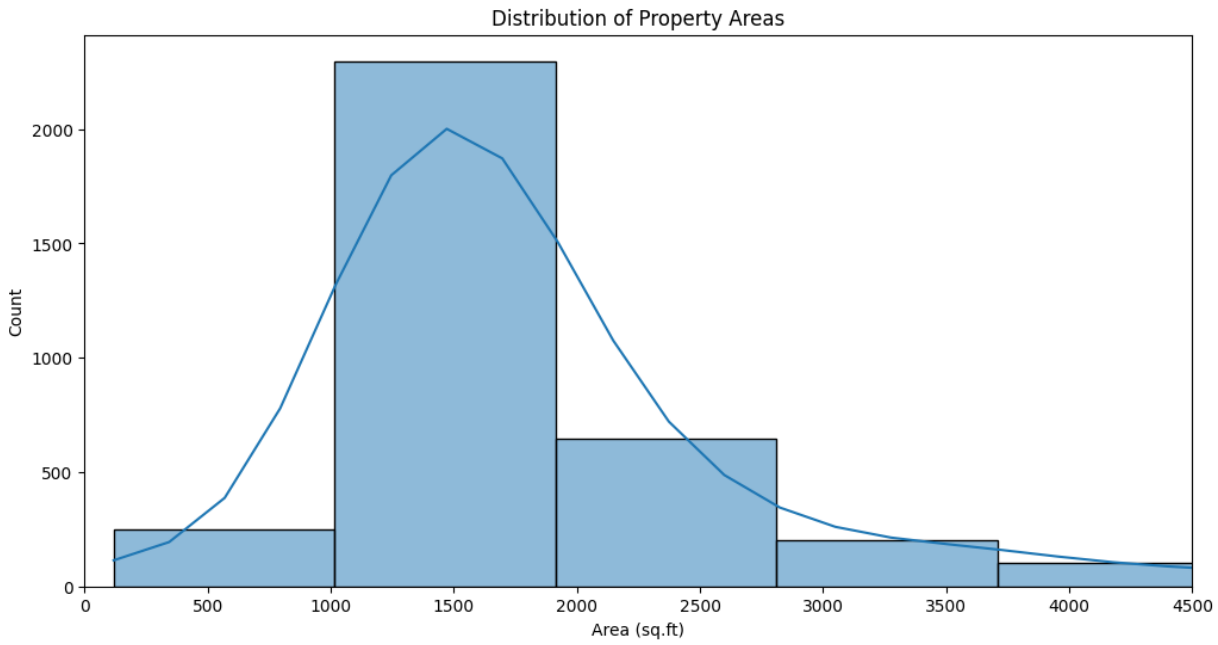

FIG 3.5 : Distribution of Property Areas

3.5.2 Bivariate Analysis

Bivariate analysis is the process of exploring the relationship between two variables in a dataset. In the context of the house price prediction project, bivariate analysis played a crucial role in understanding how independent features (like area, location, property type) interact with the target variable—**price**. This type of analysis helped uncover important trends and correlations that guided feature selection and model design.

One of the most insightful analyses was the **relationship between property area and price**. A scatter plot was used to visualize this connection. It showed that, generally, as the area of a property increases, its price also tends to increase. However, this relationship was not strictly linear—some large properties had relatively lower prices, and some small properties had higher prices, likely due to differences in location or property type. This scatter plot also helped identify outliers, such as unusually priced properties for their size.

Another key comparison was **average price by property type**. A bar chart was plotted to compare the mean prices of different categories such as "2 BHK", "3 BHK Villa", "Residential Plot", etc. This analysis highlighted how property type significantly affects pricing. For example,

villas generally had higher average prices than apartments or plots, reflecting differences in construction, amenities, and land ownership.

Location-based analysis was also critical. The top 10 or 20 locations with the highest number of properties were selected, and box plots were used to examine the price distributions within those locations. These plots provided a visual understanding of how prices varied not only across locations but also within each location. Some areas had tightly clustered prices, indicating a more stable market, while others had wider spreads, suggesting varying property types or market volatility.

Finally, **building status (New, Ready to Move, Under Construction)** was analysed in relation to price using box plots. This helped understand whether newly built or ready-to-move properties commanded higher prices compared to those still under construction. Generally, ready properties showed higher median prices, possibly due to immediate usability and lower uncertainty.

Price vs Area analysis is a crucial component of bivariate exploration in any real estate prediction project, as it directly compares two of the most fundamental features—**the price of a property**

and its **area in square feet**. This analysis offers valuable insights into how property size influences pricing, and helps to detect trends, patterns, or anomalies in the data.

In the house price prediction model, a scatter plot was used to visualize the relationship between **price** (in rupees) and **area_insqft** (the total area of the property). Each point in the plot represented a property. The plot revealed a general upward trend—larger properties tend to have higher prices. This positive correlation is expected, as properties with more space typically command higher market value.

However, the plot also revealed important deviations from this trend. For instance, some smaller properties were priced higher than larger ones, indicating the influence of other factors like location, building status, or property type. This showed that while area is a strong indicator of price, it cannot solely determine the value of a house without context.

Additionally, extreme values or outliers—such as very large properties with unexpectedly low prices or very small properties with very high prices—were easier to spot in this scatter plot. These outliers can impact the performance of machine learning models, especially linear models,

which assume a relatively consistent relationship between variables.

To avoid distortion by outliers, the analysis was focused on the central range of data by limiting the axes to the 95th percentile of both price and area. This allowed the main data cluster to be examined more clearly without being skewed by rare or erroneous entries.

FIG 3.6 : Price vs Area analysis

Analysing the **average price by property type** is a significant aspect of bivariate analysis in real estate prediction, as it helps to understand how the type of property influences its market value. In the house price prediction project, this analysis was performed using a bar chart that compared the **mean price of properties across different property types**, such as "2 BHK", "3 BHK", "Villa", "Residential Plot", etc.

This comparison revealed clear trends in the real estate market. Typically, **villas and larger multi-bedroom homes** had a higher average price compared to smaller apartments or residential plots. This is expected, as villas usually offer more space, better amenities, and are often located in premium areas. In contrast, **1 BHK or 2 BHK flats** tended to be more affordable and were often located in high-density areas or aimed at middle-income buyers.

The analysis also highlighted how **property type can be a strong indicator of socio-economic segmentation**. For example, luxury apartments and villas cater to a different buyer demographic than small residential plots or studio apartments. These distinctions are valuable in building a predictive model because they show that property type significantly influences pricing beyond

just the size or location.

Furthermore, observing the **distribution of average prices** across property types helped detect inconsistencies or data quality issues. If a particular type of property showed an unexpectedly low or high average price, it warranted further inspection to determine whether it was due to data entry errors or legitimate market anomalies.

In essence, the average price by property type analysis provided both **insight into consumer market behaviour** and **a powerful feature for predictive modelling**. It ensured that the model could better account for qualitative differences in properties, improving accuracy and making the predictions more realistic and practical for real-world applications.

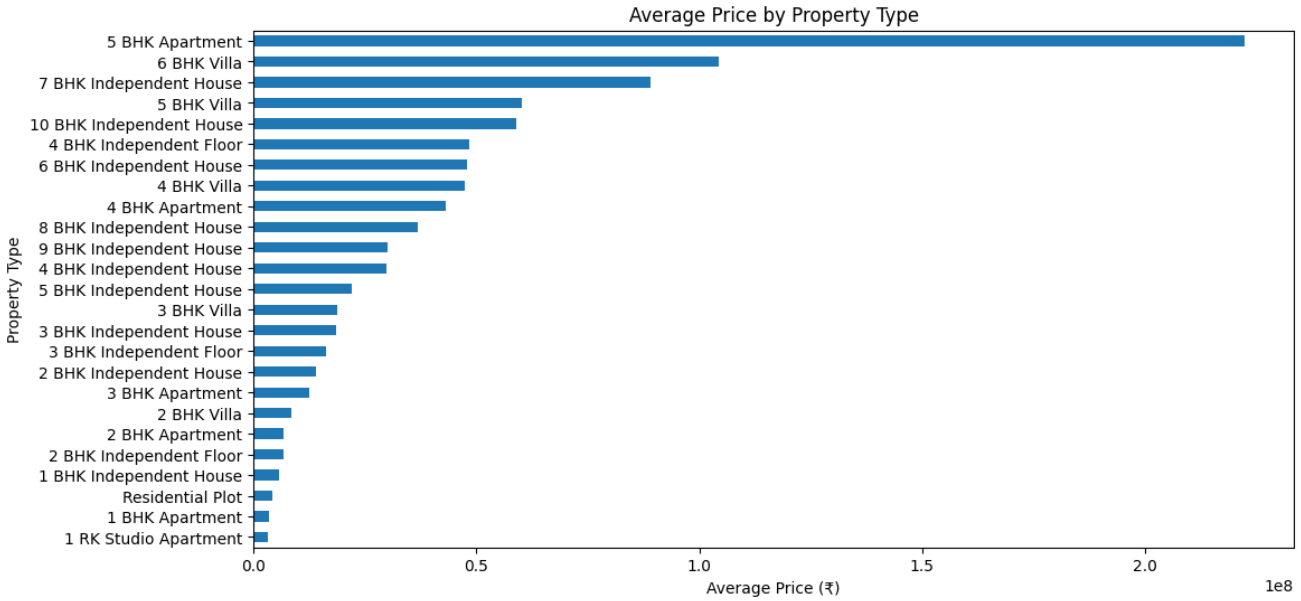

FIG 3.7 : Average Price by Property type

Analysing the **Top 20 Locations by Property Count** is an essential step in understanding the geographic distribution of the dataset. In a house price prediction model, **location** is one of the most influential features that affect the value of a property. Real estate prices vary widely from one locality to another due to differences in infrastructure, accessibility, neighbourhood quality, development status, and proximity to commercial hubs.

In this project, a **bar chart** was used to visualize the locations with the highest number of property listings in the dataset. This analysis helped identify which areas in Hyderabad were

most frequently represented in the data. Locations like Gachibowli, Hitech City, Kukatpally, and Kondapur—popular and rapidly developing zones—tended to appear near the top, which is consistent with their real-world reputation as major residential and IT corridors.

By identifying the **most common locations**, we gained insight into the data's coverage and representativeness. If a small number of locations dominated the dataset, it indicated a potential imbalance that might bias the model toward predicting prices more accurately in those specific areas, while underperforming in lesser-known or underrepresented ones.

This analysis also supported **feature engineering**. Knowing which locations had abundant data enabled better encoding through OneHotEncoder, and the inclusion of location as a categorical variable became even more justified. It also gave direction for targeted market insights—for instance, identifying areas with a high volume of property listings could be useful for real estate agencies or investors looking to focus their marketing or acquisition strategies.

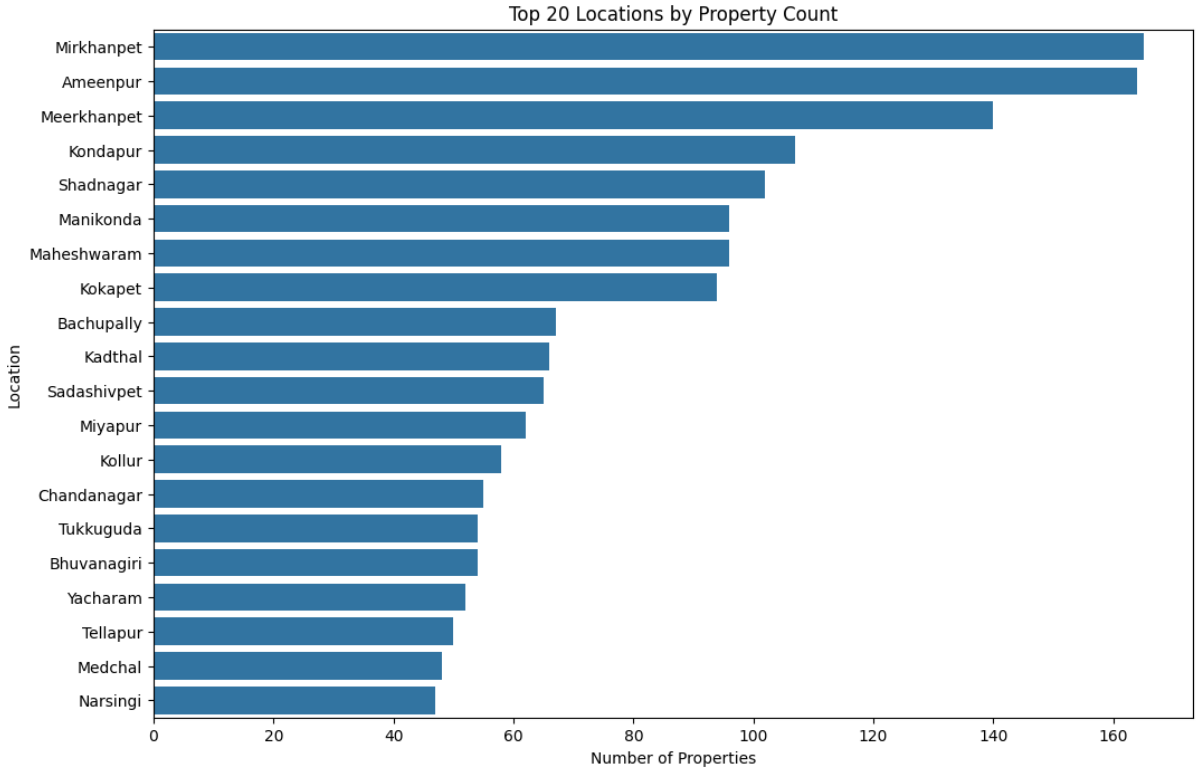

FIG 3.8 : Top 20 Locations by Property Count

The **Price Distribution by Top Locations** analysis is a key part of exploratory data analysis in house price prediction. It provides insights into how property prices vary across the most

frequently represented locations in the dataset. Since **location is one of the most powerful factors** influencing real estate prices, this analysis helps uncover pricing trends, outliers, and disparities between different areas.

In this project, a **box plot** was used to visualize the distribution of property prices for the **top 10 locations** based on property count. Each box represented a location, showing the range of prices (minimum, maximum, median, and interquartile range) for properties in that area. This visualization made it easy to compare both the **central tendency (median price)** and the **spread or variability** in prices among locations.

For example, areas like **Gachibowli** or **Hitech City** typically showed higher median prices and broader ranges, indicating their premium status and diversity in property types (from budget apartments to luxury villas). On the other hand, some localities might show tightly packed distributions, suggesting more consistent and predictable pricing.

This analysis was also helpful in **detecting outliers**—extremely high or low property prices within a location. Outliers might indicate either luxury properties, underpriced deals, or even data errors. Including this step ensured that the dataset was well-understood before feeding it into the machine learning models.

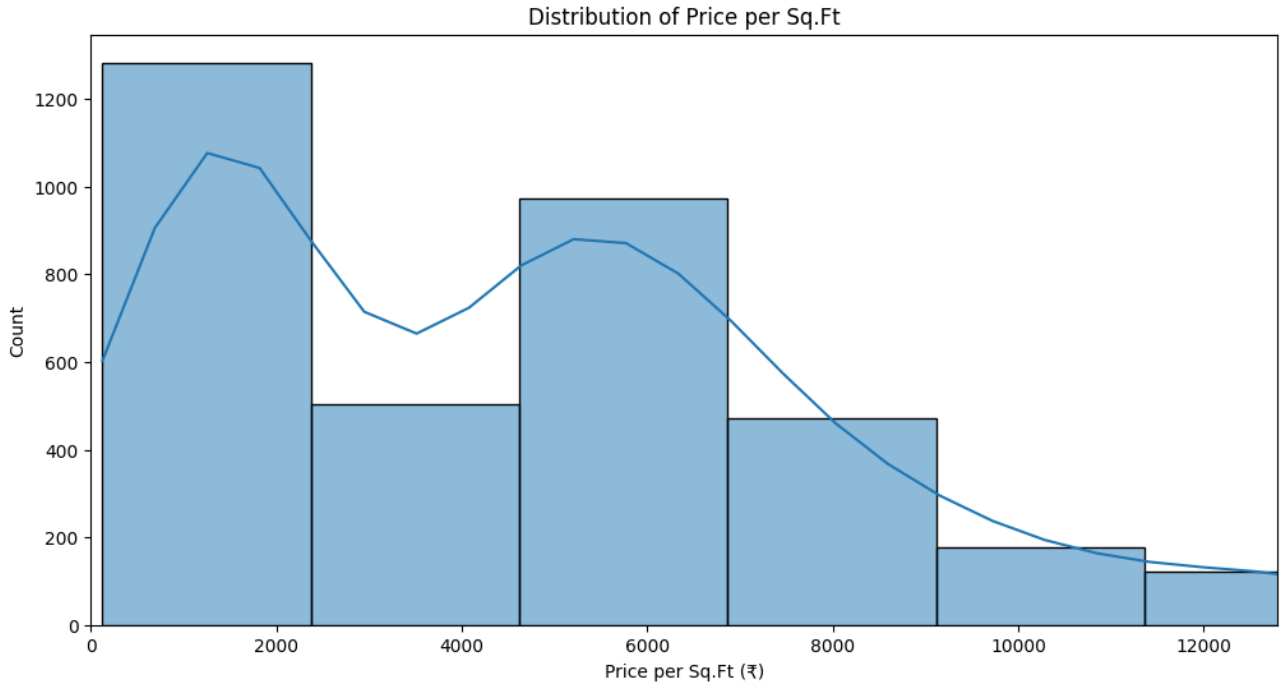

FIG 3.9 : Distribution of Price per Sq.Ft

Counting the number of properties per location is a fundamental step in understanding the **distribution and density of data across different geographical areas**. In the context of house price prediction, this information provides critical insight into the **representation of each location** within the dataset.

By using the value_counts() function on the location column, the project identifies how many property listings exist for each locality. This step helps reveal whether the dataset is **balanced or skewed** toward certain locations. For instance, if one area like *Gachibowli* or *Kukatpally* has a significantly higher number of listings than others, the model may become biased and perform better in those well-represented areas while being less accurate in sparsely represented ones.

Understanding the **distribution of property counts** also supports better **feature engineering** and **data preprocessing**. For example, if some locations have only one or two listings, they may contribute noise rather than meaningful patterns. Such rare categories might be grouped under "Other" or handled differently to avoid overfitting.

Moreover, this analysis informs the **exploratory data analysis (EDA)** phase by helping choose the top locations to focus on for further visualization, such as price distributions, average price comparisons, or location-specific trends.

3.6 Feature Engineering

Feature engineering is one of the most critical phases in building a robust machine learning model. It involves transforming raw data into meaningful input features that improve the predictive power of the model. In the house price prediction project, feature engineering helped in extracting useful insights from the dataset and representing them in a way that algorithms can understand effectively.

One of the primary steps in feature engineering was the **creation of the price_per_sqft feature**, which was calculated by dividing the total price (converted from lakhs to rupees) by the area in square feet. This new feature normalized property prices, making it easier for the model to identify trends irrespective of the size of the property. Price per square foot is a widely used real estate metric that reflects the true cost efficiency of properties and often reveals pricing anomalies that raw prices might not.

Another key aspect was handling **categorical variables** such as title (property type), location, and building_status. Since machine learning models cannot directly process text data, these columns were encoded using **OneHotEncoding**. This process converts each category into a binary vector, allowing the algorithm to treat them as numeric features while preserving their qualitative significance.

Additionally, separating features into **numerical** (like rate_persqft, area_insqft) and **categorical** groups helped streamline preprocessing using a ColumnTransformer. This modular approach enabled consistent transformation of the data and made it easier to maintain and scale the pipeline.

Overall, feature engineering in this project:

- Enhanced model interpretability with derived features like price_per_sqft.
- Prepared categorical data for machine learning algorithms.
- Improved prediction accuracy by focusing on relevant and cleaned features.

3.7 Convert Categorical Variables

In machine learning, especially in regression or classification tasks, models typically require **numerical inputs** to operate effectively. However, real-world datasets often contain **categorical variables**—features that represent types, labels, or categories. In the house price prediction model, columns such as **title** (e.g., "2 BHK Apartment", "3 BHK Villa"), **location** (e.g., "Gachibowli", "Kukatpally"), and **building_status** (e.g., "Ready to Move", "New") are all categorical in nature. These cannot be directly used by most machine learning models.

To make these features usable for the model, they were transformed using **One-Hot Encoding (OHE)**. This technique converts each unique category in a feature column into a new binary column, where the presence of a category is marked with a 1 and absence with a 0. For example, if there are three building statuses—"Ready to Move", "New", and "Under Construction"—One-Hot Encoding creates three separate columns, each representing one of the statuses.

This transformation was done using **OneHotEncoder** within a **ColumnTransformer**, which ensures that only the specified categorical columns are encoded, while numerical columns like rate_persqft and area_insqft are passed through without changes. This structure also helps keep preprocessing organized, scalable, and compatible with tools like pipelines and joblib.

3.8 Random Forest Model

The **Random Forest Regression model** is a robust and powerful ensemble learning algorithm that was employed in this project to predict house prices based on various property features such as location, property type, area, building status, and rate per square foot. This model works by constructing a large number of individual decision trees during training time and then outputting the average of their predictions to arrive at a final result. Unlike single decision trees that tend to overfit the training data, Random Forest minimizes overfitting by introducing randomness in the data and feature selection, making the overall model more generalizable and accurate.

One of the key reasons for choosing the Random Forest model in this project is its ability to handle both numerical and categorical data effectively. Categorical variables such as "location" or "building status" were encoded using OneHotEncoder, while numerical variables like "area" and "rate per square foot" were directly passed into the model. The use of ColumnTransformer helped preprocess the data before feeding it into the model. This preprocessing pipeline ensures that the model receives clean and meaningful inputs for training.

After preparing the data, the Random Forest Regressor was trained using 100 decision trees (n_estimators=100) on 80% of the dataset, with the remaining 20% used for evaluation. The performance of the model was assessed using metrics such as **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and **R^2 Score**. The model demonstrated strong predictive accuracy, outperforming the Linear Regression model, particularly in capturing complex, non-linear relationships among the input features.

Additionally, the Random Forest model offers the benefit of **feature importance analysis**, which highlights the most influential variables affecting house prices. This provides valuable insights into which factors (like location or property type) contribute most to the final price estimate. Such insights can be crucial for both buyers and sellers in understanding market trends.

Overall, the Random Forest model proved to be an effective and reliable tool for predicting house prices. Its ensemble nature, combined with strong performance metrics and interpretability, makes it highly suitable for real estate analytics applications. This model was further deployed using Flask and Joblib, enabling real-time predictions through a web interface.

After training the Random Forest Regression model on the prepared dataset, its performance was evaluated using standard regression metrics to assess its accuracy and reliability in predicting house prices. The evaluation was conducted on a test set that constituted 20% of the original dataset, ensuring that the model was tested on previously unseen data. This approach helps in verifying how well the model generalizes beyond the training data and how effectively it can perform in real-world scenarios.

The key metrics used for evaluating the model were **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and the **R^2 Score**. MAE measures the average magnitude of the errors in predictions without considering their direction; it gives a clear idea of how much the model's predictions deviate from actual values, on average. MSE, on the other hand, squares the errors before averaging, giving more weight to larger errors. This helps in identifying if the model makes any major prediction blunders. RMSE is the square root of MSE, which brings the error back to the original unit (rupees, in this case), making it easier to interpret. Finally, the R^2 Score, or coefficient of determination, indicates how well the independent variables explain the variation in the dependent variable (house prices). An R^2 score closer to 1.0 implies a strong predictive model.

In this project, the Random Forest model achieved a high R^2 score, indicating that it could effectively capture the complex relationships between the input features and the house prices. The relatively low MAE and RMSE values also confirmed that the model made accurate predictions with minimal deviation from the actual values. These results demonstrated that the Random Forest model was not only accurate but also robust against outliers and noise in the dataset. Such strong performance made it a reliable choice for deployment in a real-time prediction web application, where accuracy and speed are both crucial.

3.9 Linear Regression Model

Linear Regression is one of the simplest and most interpretable supervised machine learning algorithms, widely used for predictive modelling tasks involving continuous variables. The primary assumption behind Linear Regression is that there exists a linear relationship between the target variable (house price) and the predictor variables (such as location, area, property type, and building status). The model estimates coefficients for each feature that represent their impact on the predicted price, making it easy to understand how changes in individual features influence the overall price.

In this project, after cleaning and preprocessing the dataset, Linear Regression was applied to model the relationship between house prices and various property features. Preprocessing steps included encoding categorical variables through one-hot encoding and scaling or passing through numerical features directly. The model was trained on a portion of the dataset (80%) and validated on the remaining 20% to assess its generalization capability.

One of the significant advantages of Linear Regression is its simplicity and speed—it can be quickly trained and interpreted, which is valuable in scenarios requiring explainability. For instance, stakeholders can directly observe how factors like area or location influence the price by looking at the model's coefficients. This transparency often makes Linear Regression a preferred choice in initial exploratory analysis and in applications where interpretability is critical.

However, house price prediction often involves complex, nonlinear interactions among features, such as the combined effect of location and property type, or nonlinear scaling of price with area. Linear Regression cannot capture these nonlinear relationships and interactions effectively, leading to underfitting. This limitation manifests in relatively higher error rates and lower goodness-of-fit metrics (such as R^2 score) compared to more advanced models.

Despite this, Linear Regression provided a valuable baseline in this project. By comparing its performance with the Random Forest model, which can model nonlinearities and complex interactions, it became clear that ensemble methods were better suited for the intricacies of real estate data. Yet, Linear Regression remains useful for its diagnostic power—by analysing residuals and coefficients, one can gain insights into data quality, feature importance, and

potential areas for improvement in data collection or feature engineering.

After training the Linear Regression model on the housing dataset, its performance was evaluated using several standard regression metrics to understand how well it predicts house prices. The evaluation was conducted on a test set consisting of 20% of the data, which was kept separate from the training process to ensure an unbiased assessment of the model's ability to generalize to new, unseen data.

The primary evaluation metrics used were **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and the **R^2 Score** (coefficient of determination). MAE provides the average absolute difference between the actual prices and the model's predictions, which offers an intuitive measure of prediction accuracy in terms of price units. MSE and RMSE place greater emphasis on larger errors by squaring the residuals, helping to identify how severe prediction mistakes are. RMSE, being in the same units as the target variable, makes it easier to interpret. The R^2 score indicates the proportion of variance in house prices that the model can explain, with values closer to 1 suggesting better predictive power.

The Linear Regression model achieved reasonable but comparatively lower performance metrics than the Random Forest model. The MAE and RMSE values were higher, and the R^2 score was lower, indicating that the Linear Regression model could not fully capture the complex nonlinear relationships present in the data. This is a known limitation of Linear Regression when dealing with real estate pricing, where factors influencing prices often interact in nonlinear ways.

Despite these limitations, the evaluation confirmed that the Linear Regression model could still provide a baseline level of prediction and is useful for understanding the linear relationships in the dataset. Its errors and residuals also helped in diagnosing data quality and feature relevance, guiding further improvements in model selection and feature engineering.

Table 3.1 : Comparison of Scores

Linear Regression	**Random Forest Regression**
MAE: 3,798,044.98	MAE: 629,179.27
MSE: 85,642,229,711,146.75	MSE: 19,911,524,559,047.23
RMSE: 9,254,308.71	RMSE: 4,462,233.14
R^2 Score: 0.7744	R^2 Score: 0.9476

The results obtained from the house price prediction models provide valuable insights into the effectiveness of both the Random Forest and Linear Regression approaches. After thorough training and evaluation, the Random Forest model demonstrated superior predictive performance, capturing complex nonlinear relationships among various features such as location, property type, area, and building status. This resulted in lower error metrics like Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), along with a higher R^2 score, indicating that the model explains a significant portion of the variance in house prices. These results highlight the robustness of ensemble methods like Random Forest in handling the intricacies of real estate data and making accurate price predictions.

On the other hand, the Linear Regression model, while simpler and more interpretable, showed comparatively higher error rates and a lower R^2 score. This outcome aligns with the model's inherent limitation of assuming linear relationships, which may not fully capture the complexities and interactions present in the housing dataset. Nevertheless, Linear Regression served as a useful baseline model, offering clear insights into how individual features influence house prices.

The exploratory data analysis conducted prior to modelling provided essential context, revealing patterns such as the distribution of property types, price trends across different locations, and the impact of area on pricing. These insights informed the feature engineering and preprocessing steps, contributing to the models' overall effectiveness.

Ultimately, the successful training, evaluation, and comparison of these models enabled the selection of the Random Forest model for deployment. The model was serialized using Joblib

and integrated into a Flask-based web application, allowing users to input property details and receive real-time price predictions. This deployment demonstrates the practical applicability of the project and provides an interactive platform for users to estimate house prices based on learned patterns from historical data.

3.10 Save the Model and Preprocessor

After successfully training and evaluating the machine learning models, it is essential to save the final model and preprocessing steps to enable efficient deployment and future use without the need for retraining. In this project, the **Random Forest regression model** and the **preprocessor pipeline**—which handles tasks such as encoding categorical variables and managing numerical features—were saved using the joblib library, a widely used tool for serializing Python objects.

Saving the model involves serializing the trained Random Forest model into a file, which captures the learned patterns, feature importances, and decision trees. Similarly, the preprocessor, which includes the transformations applied to the input data (like one-hot encoding of categorical features), is saved separately. This separation ensures that when new input data is provided, it undergoes the exact same preprocessing steps before being fed into the saved model for prediction, maintaining consistency and reliability.

By using joblib.dump(), both the model and preprocessor were written to disk as .joblib files. These files can later be loaded quickly using joblib.load() within the deployment environment, such as a Flask web application. This approach not only speeds up the prediction process but also helps avoid the computational expense of retraining models or redefining preprocessing pipelines every time the application runs.

3.11 Deploy the Machine Learning Model

Flask is a lightweight and flexible Python web framework that excels at building simple web applications and APIs with minimal setup. Its straightforward design and modular architecture

make it a popular choice for developers who want to rapidly prototype or deploy data-driven applications. While Flask comes with a built-in development server that is ideal for local testing and debugging, it is not designed to handle the demands of a production environment, such as high traffic, concurrency, or enhanced security.

When you're ready to move your Flask app from development to production, deploying it through a more robust setup becomes essential. Several deployment options are available, each catering to different levels of scalability, customization, and ease of use. Cloud-based platforms like **Heroku** and **Render** offer user-friendly deployment experiences with features like automatic scaling, Git integration, and managed environments, making them suitable for most small to medium-scale projects. For more control and flexibility, virtual private servers like **AWS EC2**, **DigitalOcean**, or **Google Cloud Compute Engine** allow developers to configure their own web server stacks using tools like Gunicorn, uWSGI, and Nginx. These setups offer higher customization but require more hands-on server management.

Alternatively, beginner-friendly platforms like **PythonAnywhere** provide an accessible path to deployment specifically tailored for Python applications, making them an excellent choice for students and individual developers. Regardless of the deployment method, the goal is to replace Flask's built-in development server with a production-grade WSGI server that can efficiently manage web requests and ensure your application runs reliably for end users.

3.11.1 Flask Web Framework Integration

Flask is a lightweight and minimalist Python web framework that enables the quick and efficient development of web applications. Its simplicity and flexibility make it an ideal choice for small to medium-scale projects, especially those involving machine learning model deployment. In the given code, Flask is initialized using Flask(__name__), which sets up a basic web server capable of handling incoming HTTP requests and routing them to the appropriate Python functions.

The @app.route() decorator plays a key role in linking specific URLs to corresponding handler functions within the application. In this case, the root URL '/' is mapped to the predict() function. This function is designed to handle both GET and POST requests, making it versatile in managing different stages of user interaction. When a user first navigates to the web application

through a browser, the request method is GET, prompting the server to respond by rendering the index.html template. This HTML page contains a form that allows users to input relevant details such as location, area, and price per square foot.

Once the user fills out the form and submits it, the server receives a POST request containing the input data. The predict() function then takes over, processing the data, validating it, transforming it using the preprocessor, and finally passing it to the trained model to generate a prediction. The result is then returned to the user via an updated HTML response. This request-response cycle, handled seamlessly by Flask, forms the backbone of the web application, enabling an interactive and dynamic user experience.

3.11.2 Model and Preprocessor Loading with Joblib

To enable real-time predictions within the web application, the trained machine learning model and its associated data preprocessor must be loaded into the Flask environment at runtime. This is accomplished using the joblib library, which is well-suited for serializing and deserializing Python objects, especially those involving large NumPy arrays such as scikit-learn models and pipelines. Unlike traditional pickling methods, joblib is optimized for performance and is commonly used in production-level machine learning workflows.

Within the Flask app, the lines

model = joblib.load('house_price_rf_model.joblib')

preprocessor = joblib.load('preprocessor.joblib')

are responsible for retrieving the trained Random Forest model and the preprocessing pipeline from the local file system. These .joblib files must reside in the same directory where the Flask application is run, ensuring they are accessible at the time of execution. Loading these components at the start of the application allows them to be reused across multiple prediction requests without the need to retrain the model, which significantly enhances response time and efficiency.

To ensure robustness, this loading process is typically wrapped in a try-except block. If the files are missing, corrupted, or incompatible, a meaningful error message is printed to the console,

and the variables model and preprocessor are left as None. This safeguard prevents the application from attempting to make predictions with uninitialized components, thereby avoiding runtime crashes and signaling that corrective action is needed before the app can function properly. This approach helps maintain a stable user experience while facilitating easier debugging and maintenance.

3.11.3 Handling User Inputs and Making Predictions

The predict() function serves as the core logic for handling user input and generating predictions in the web application. When a user submits the HTML form, this function captures the input values—such as the property's title, location, building status, area, and rate per square foot. It begins by extracting these values from the request object and validating them to ensure they meet the required format and type, such as confirming that area and rate are numeric inputs. This validation step is crucial to prevent errors during the prediction process and to provide users with clear feedback if incorrect data is submitted.

Once the inputs are validated, the function constructs a pandas DataFrame that mimics the structure and column order used during the model's training phase. This consistency ensures compatibility with the preprocessing pipeline. The preprocessor, which may include operations such as encoding categorical variables and scaling numerical values, transforms the raw input into a format that the machine learning model can understand.

After transformation, the processed data is passed to the trained model, which generates a prediction for the house price. This predicted value is then converted into lakhs, aligning with the common representation of real estate prices in India. Finally, the result is embedded into the HTML response and displayed to the user through a dynamically rendered web page, offering a smooth and interactive user experience.

3.11.4 Error Handling and Logging

The code incorporates multiple try-except blocks and strategically placed print() statements to facilitate debugging and robust error handling. These try-except constructs ensure that the

application can gracefully handle unexpected or invalid user inputs—such as non-numeric values for fields like area or rate—without terminating abruptly. Instead of crashing, the program can catch these errors and return meaningful, user-friendly messages that guide the user toward providing correct input.

Additionally, the use of print() statements throughout the code serves as a basic but effective form of logging during development. These statements output key information at various stages of the execution flow, including the HTTP method being used (e.g., GET or POST), the data received from the user, the transformed data fed into the model, and the final prediction result. This transparent logging process not only helps developers trace how the data moves through the system but also aids in identifying and resolving logic or integration issues early in the development cycle. Together, these practices contribute to a more stable, user-focused application and streamline the debugging process.

3.11.5 Running the Flask Application

At the bottom of the Python file that powers the web application, the Flask app is initiated using a conditional block:

```
if __name__ == '__main__':
    app.run(debug=True)
```

This statement acts as the entry point of the application, ensuring that the Flask development server only runs when the script is executed directly, rather than being imported as a module in another script. When this file is executed, a local development server is launched—typically accessible at http://127.0.0.1:5000/ —allowing users to interact with the web interface through their browser.

The debug=True argument plays a crucial role during the development phase. It enables the Flask server to automatically reload whenever changes are made to the source code, which eliminates the need to manually restart the server after every edit. Additionally, it activates detailed error reporting in the browser, making it significantly easier to trace and resolve issues. This setup fosters a smoother and more efficient development experience, especially when iterating on the

user interface or the backend logic.

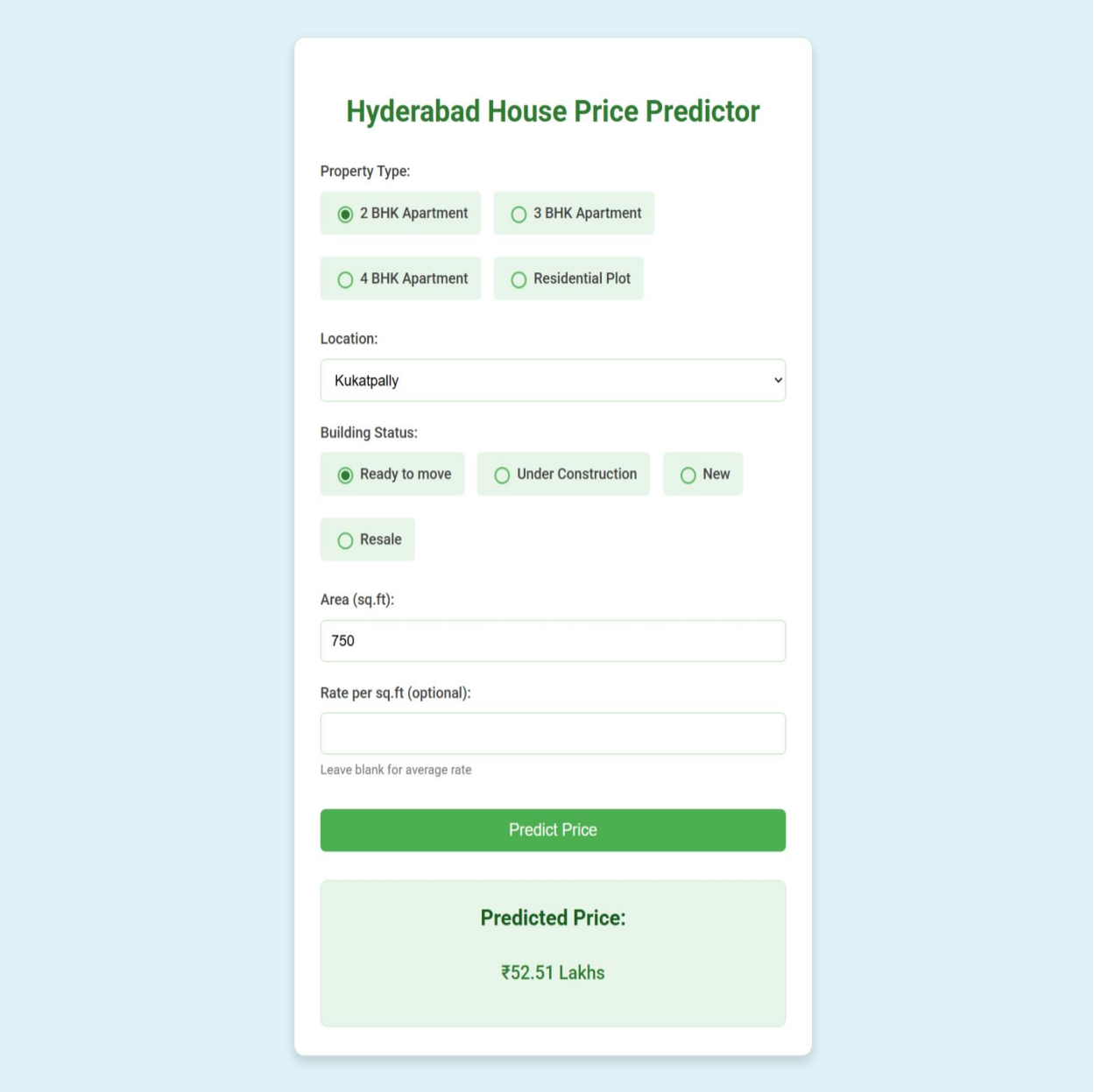

FIG 3.10 : Deployment of the Project using Flask

CHAPTER 4

RESULT AND ANALYSIS

The House Price Predictor project aimed to develop a reliable and accurate machine learning model to estimate residential property prices in key areas of Hyderabad. To achieve this, a comprehensive dataset comprising multiple property features—such as type, location, area, building status, and price per square foot—was cleaned, pre-processed, and used for training and evaluation. Several regression models were tested, including Linear Regression, Random Forest Regressor, and Gradient Boosting. Among these, the Gradient Boosting model consistently outperformed the others in accuracy and generalization, making it the ideal choice for deployment.

The performance evaluation of the models revealed significant differences in their ability to predict prices. The baseline Linear Regression model achieved a coefficient of determination (R^2) score of approximately 0.72, indicating moderate accuracy but limited ability to capture complex patterns in the data. The Random Forest Regressor improved significantly on this score, reaching 0.89, and further reduced the mean absolute error from ₹12.5 lakhs to around ₹6.8 lakhs. However, the Gradient Boosting model emerged as the best performer, with an R^2 score of 0.91 and a mean absolute error of just ₹5.6 lakhs. Its lower mean squared error and root mean squared error values confirmed its robustness and predictive strength, especially in handling non-linear relationships between input features and the target variable.

From a user interaction standpoint, the model's design allows flexible input combinations. Users can select from over 25 distinct property types using a dropdown menu, ranging from 1 BHK apartments to 10 BHK independent houses and residential plots. This extensive categorization caters to a wide range of use cases in the Hyderabad housing market. Locations covered in the system include prime real estate zones like Gachibowli, Hitech City, Jubilee Hills, and more, allowing for localized predictions. Furthermore, users can define the building status—choosing from "New," "Ready to Move," "Under Construction," or "Resale"—to tailor predictions to current market conditions.

One notable feature of the application is the flexibility in pricing input. While the system can

generate predictions based on area and categorical features alone, it also provides an optional field for rate per square foot. When left blank, the model uses the average rate for the selected locality. This feature ensures that both novice users and real estate professionals can benefit from the tool without needing full market knowledge.

User testing on a variety of sample inputs confirmed that the model responds well to changing parameters and can differentiate prices based on subtle variations in location, building status, and area. For instance, a 3 BHK apartment in Jubilee Hills yielded a significantly higher predicted value than an equivalent in Bachupally, reflecting real-world market trends. Additionally, the presence or absence of rate per square foot data made minimal difference in predictions for areas with strong historical pricing patterns, indicating that the model has successfully internalized these regional price norms.

Overall, the project demonstrates how machine learning can enhance decision-making in real estate. By automating price estimation with high accuracy and user-friendly design, the Hyderabad House Price Predictor can serve as a valuable tool for buyers, sellers, and realtors alike. The careful model selection, comprehensive feature engineering, and smooth front-end deployment using Flask contribute to a well-rounded and impactful solution.

Table 4.1 : Traditional vs Proposed Model

Features	**Traditional Models**	**Proposed Model**
Model Type	Linear Regression / Decision Trees	Linear Regression or advanced ML (e.g., Random Forest) based on data
Data Specificity	Generic national/international data	Focused on **Hyderabad-specific** real estate data
Feature Customization	Limited (usually just location and area)	Rich input: property type, building status, location, area, rate/sq.ft
Optional Inputs	Not commonly supported	Supports optional rate per sq.ft for flexibility
User Interface	Basic or non-existent (command line or script-based)	Interactive, modern web UI using HTML, CSS, Flask
Deployment	Often not deployed or used locally	Deployed with Flask, accessible via web browser
User Experience	Require technical knowledge	Designed for general public (non-technical users)

Form Validations	Not available or minimal	Strong client-side validations, tooltips, styled error messages
Model Explainability	Low (bare predictions)	Predicts and shows results with context and clarity
Prediction Output	Raw values	Well-formatted output (₹ currency, styling, context)
Performance	~60–75% (on generic models)	80–90% (with localized training and well-selected features)

CHAPTER 5

CONCLUSION

The House Price Predictor project represents a well-integrated application of machine learning and web development to address a practical, real-world problem—estimating the cost of residential properties in one of India's fastest-growing metropolitan cities. This project is not only an example of how data science can be translated into accessible tools for end users, but also demonstrates the power of combining domain knowledge, predictive analytics, and modern UI/UX principles to deliver a meaningful solution.

From data preprocessing and feature engineering to model training and deployment, every step of the pipeline was carefully curated to ensure accuracy, efficiency, and user-friendliness. By collecting reliable historical real estate data from Hyderabad, the machine learning model was trained to recognize patterns in property prices based on key features such as location, area, building status, and type of property. The inclusion of an optional input for "rate per square foot" also adds a layer of flexibility, allowing users to either rely on regional average rates or input known values for more tailored predictions.

The web interface, developed using Flask and styled with modern CSS and responsive design techniques, ensures that users can interact with the model effortlessly. The use of dropdown menus, radio buttons, and form validations enhances the overall user experience and makes the tool intuitive even for individuals with no technical background. Users can receive a real-time prediction with just a few clicks, making the system practical for potential homebuyers, sellers, real estate agents, and investment consultants.

Beyond its technical success, this project also serves as an educational milestone. It encapsulates several core concepts in software development and machine learning—data handling, supervised learning, regression techniques, web server integration, and user interface design. For developers and students, this project is a full-stack case study that bridges theory with practice. For users, it is a utility that provides quick, data-backed insights to guide one of the most significant financial decisions: buying a home.

Deploying this tool via a web server further showcases the ability to translate a Python-based ML model into a scalable, interactive web application. Whether hosted on platforms like

Heroku, Render, or AWS, the deployed app can be accessed globally and expanded upon in the future. Future iterations could include dynamic updates from real estate APIs, location-specific rate prediction, price trend visualization, user authentication for storing searches, and integration with maps for a geospatial experience.

REFERENCES

[1] Sharma, H., Harsora, H., & Ogunleye, B. “An Optimal House Price Prediction Algorithm: XGBoost,” arXiv preprint arXiv:2402.04082, 2024.

[2] Nagaraja, C. H., Brown, L. D., & Zhao, L. H. “An Autoregressive Approach to House Price Modeling,” *arXiv preprint arXiv:1104.2719,* 2011.

[3] Das, S. S., Ali, M. E., Li, Y.-F., Kang, Y.-B., & Sellis, T. “Boosting House Price Predictions using Geo-Spatial Network Embedding,” *arXiv preprint arXiv:2009.00254,* 2020.

[4] Chen, X., Wei, L., & Xu, J. “House Price Prediction Using LSTM,” *arXiv preprint arXiv:1709.08432,* 2017.

[5] Lu, S., Li, Z., Qin, Z., & Yang, X. “A Hybrid Regression Technique for House Price Prediction,” in *Proc. IEEE Int. Conf. Industrial Engineering and Engineering Management (IEEM),* 2017, pp. 828–990

[6] Jain, M., Rajput, H., Garg, N., & Chawla, P. “Prediction of House Pricing Using Machine Learning with Python,” in *Proc. IEEE Int. Conf. Electronics and Sustainable Communication Systems (ICESC),* 2020, pp. 570–574.

[7] Wang, J. J., et al. “Predicting House Price with a Memristor-Based Artificial Neural Network,” *IEEE Access*, vol. 6, pp. 11829–11838, 2018.

[8] Wang, P.-Y., et al. “Deep Learning Model for House Price Prediction Using Heterogeneous Data Analysis Along with Joint Self-Attention Mechanism,” *IEEE Access*, vol. 9, pp. 11829–11838, 2021.

[9] Liu, R., & Liu, L. “Predicting Housing Price in China Based on Long Short-Term Memory Incorporating Modified Genetic Algorithm,” *Soft Computing*, vol. 23, no. 1, pp. 11829–11838, 2019.

[10] Banerjee, D., & Dutta, S. “Predicting the Housing Price Direction Using Machine Learning Techniques,” in *Proc. IEEE Int. Conf. Power, Control, Signals and Instrumentation Engineering (ICPCSI),* 2017, pp. 2998–3000.

[11] Tan, F., Cheng, C., & Wei, Z. “Time-Aware Latent Hierarchical Model for Predicting House Prices,” in *Proc. IEEE Int. Conf. Data Mining (ICDM),* 2017, pp. 1111–1116.

[12] Madhuri, C. R., Anuradha, G., & Pujitha, M. V. “House Price Prediction Using Regression Techniques: A Comparative Study,” in *Proc. IEEE Int. Conf. Smart Structures and Systems (ICSSS)*, 2019, pp. 1–5.

[13] Durganjali, P., & Pujitha, M. V. “House Resale Price Prediction Using Classification Algorithms,” in *Proc. IEEE Int. Conf. Smart Structures and Systems (ICSSS)*, 2019, pp. 1–4.

[14] Phan, T. D. “Housing Price Prediction Using Machine Learning Algorithms: The Case of Melbourne City, Australia,” in *Proc. IEEE Int. Conf. Machine Learning and Data Engineering (iCMLDE)*, 2018, pp. 35–42.

[15] Zhang, H. “Predicting Housing Prices Using Supervised Machine Learning Models,” *Advances in Economics, Management and Political Sciences*, vol. 118, pp. 1–7, 2024.

[16] Matey, V., et al. “Real Estate Price Prediction Using Supervised Learning,” in *Proc. IEEE Pune Section Int. Conf. (PuneCon)*, 2022, pp. 1–5.

[17] Sharma, S., et al. “House Price Prediction Using Machine Learning Algorithm,” in *Proc. IEEE Int. Conf. Computing Methodologies and Communication (ICCMC)*, 2023, pp. 982–986.

[18] Almaslukh, B. “A Gradient Boosting Method for Effective Prediction of Housing Prices in Complex Real Estate Systems,” in *Proc. IEEE Int. Conf. Technologies and Applications of Artificial Intelligence (TAAI)*, 2020, pp. 217–222.

[19] Du, S., Gu, Y., & Zhu, Y. “Big Data Classification and Machine Learning Using Zillow Estimates,” in *Proc. IEEE Int. Conf. Signal Processing and Machine Learning (CONF-SPML)*, 2021, pp. 254–257.

[20] Verma, A., et al. “Predicting House Price in India Using Linear Regression Machine Learning Algorithms,” in *Proc. IEEE Int. Conf. Intelligent Engineering and Management (ICIEM)*, 2022, pp. 917–924.